

Laboratory Activity No. 1

Real-Time Multisensor Room Monitoring System

Name: Carl Hovan B. Chan	Date: September 28, 2026
Section: B187	Repository: github.com/CHCHAN-bsca/bca182-freertos-multisensor

1. Problem and Requirements

For this lab I built a room monitor on an STM32 Blue Pill using PlatformIO, the STM32Cube HAL and FreeRTOS, written in C++. It runs in the Wokwi simulator. It reads temperature, humidity, light and motion, shows one reading at a time on an OLED, sounds a buzzer when the temperature is out of range, and turns the screen off when nobody is around.

These jobs run at different speeds. The sensors are read every 2 seconds, but the encoder and the PIR should react quickly. That is why I split the work into FreeRTOS tasks instead of writing one big loop.

ID	Requirement	How I did it
FR-01	Temperature	SensorTask reads the DHT22 every 2 s
FR-02	Humidity	SensorTask reads the DHT22 every 2 s
FR-03	Ambient light	SensorTask reads the ADC and converts it to 0-100 %
FR-04	Motion	MotionTask checks the PIR every 250 ms
FR-05	OLED display	DisplayTask shows one reading at a time
FR-06	Encoder navigation	InputTask changes the page in both directions with wraparound
FR-07	Temperature alarm	AlarmTask turns on the buzzer below 18 C or above 30 C
FR-08	Activity state	MotionTask keeps the ACTIVE / INACTIVE state
FR-09	Auto inactive	No motion for 15 s makes the system INACTIVE
FR-10	Auto reactivate	Motion makes it ACTIVE again

Wokwi adaptation. All parts are Wokwi models. I tested by changing the DHT22 and light sensor values, clicking "Simulate motion" on the PIR and using the encoder arrows. The serial monitor is on USART1. The light value is a relative percentage, not lux. FreeRTOS needed a Wokwi-compatible port (Section 7).

2. System Architecture and Design

Hardware

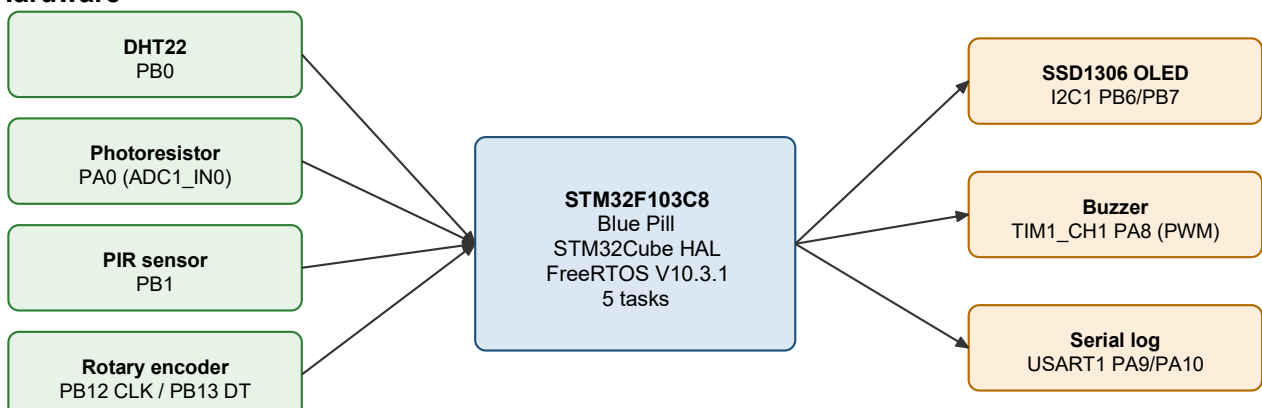


Figure 1. Hardware overview.

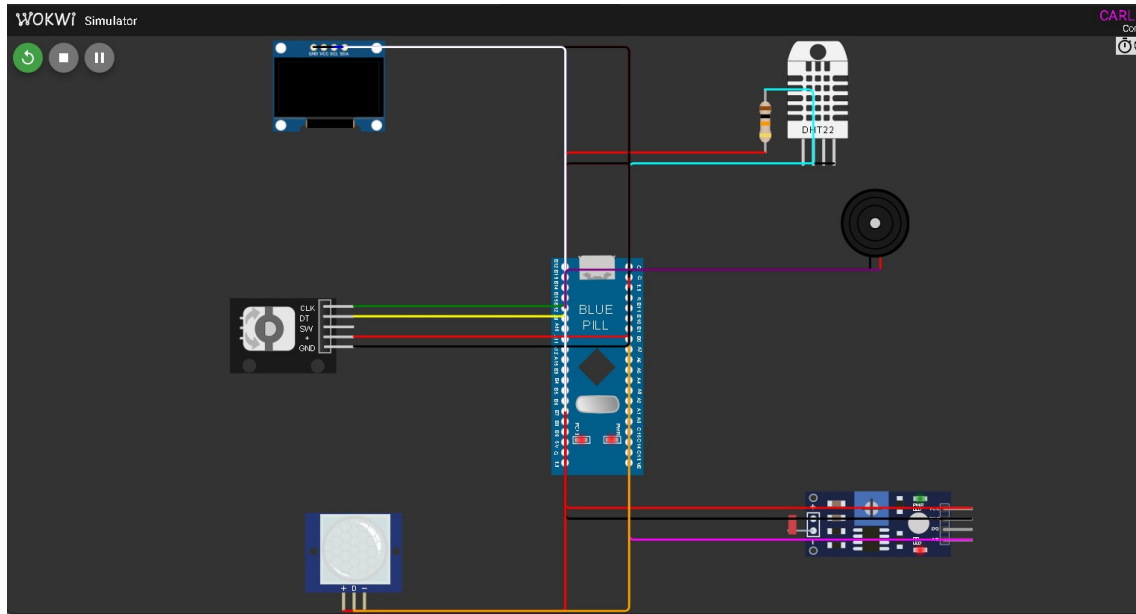


Figure 2. Wokwi circuit.

Part	Pin	Mode
DHT22	PB0	Open-drain output for the start pulse, then input with pull-up; TIM4 at 1 MHz for timing
Photoresistor	PA0	ADC1 channel 0
PIR	PB1	Digital input
Encoder CLK / DT	PB12 / PB13	Inputs with pull-up, polled by InputTask
OLED SCL / SDA	PB6 / PB7	I2C1, 400 kHz
Buzzer	PA8	TIM1 channel 1 PWM, 1 kHz
Serial TX / RX	PA9 / PA10	USART1, about 115200 baud

Timers: TIM1 for the buzzer PWM, TIM2 as the HAL time base (from CubeMX), TIM3 as the FreeRTOS tick in the Wokwi port, and TIM4 as a free-running 1 MHz counter for the DHT22 timing. The chip runs on the default 8 MHz HSI clock.

Software

File	What it does
main.cpp	Hardware init, creates the RTOS objects and tasks, starts the scheduler
hardware.cpp	GPIO, ADC, I2C, TIM4 and buzzer PWM setup
sensors.cpp	DHT22 and LDR reading, SensorTask
display.cpp, ssd1306.cpp	DisplayTask and the OLED driver
input.cpp	InputTask (encoder)
alarm.cpp, alarm_logic.cpp	AlarmTask and EvaluateTemperature()
motion.cpp, system_state.cpp	MotionTask and UpdateSystemState()
display_navigation.cpp, sensor_values.cpp	ScrollNext/ScrollPrev and the light % conversion

rtos_objects.cpp, serial_log.cpp, rtos_hooks.cpp	Queues, mutex, event group, serial output, FreeRTOS hooks
--	---

The decision logic (alarm, page navigation, light conversion, state change) is in separate files with no hardware code, so I could unit test it on my PC.

State machine

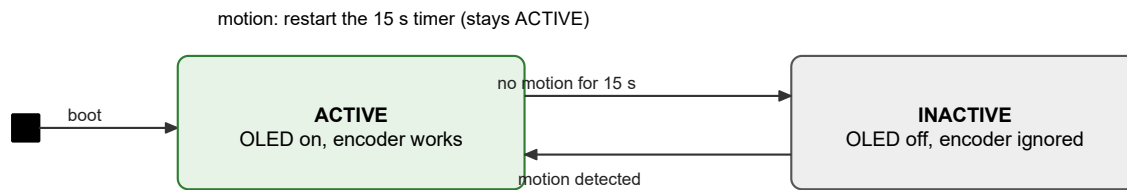


Figure 3. ACTIVE / INACTIVE state machine (*UpdateSystemState()* in *system_state.cpp*).

When ACTIVE, the OLED is on and the encoder works. When INACTIVE, DisplayTask clears and turns off the OLED and InputTask ignores the encoder. MotionTask keeps reading the PIR so motion can wake the system up. I did not make a separate StateTask; MotionTask runs the state machine because it is the task that sees the motion.

3. FreeRTOS Architecture

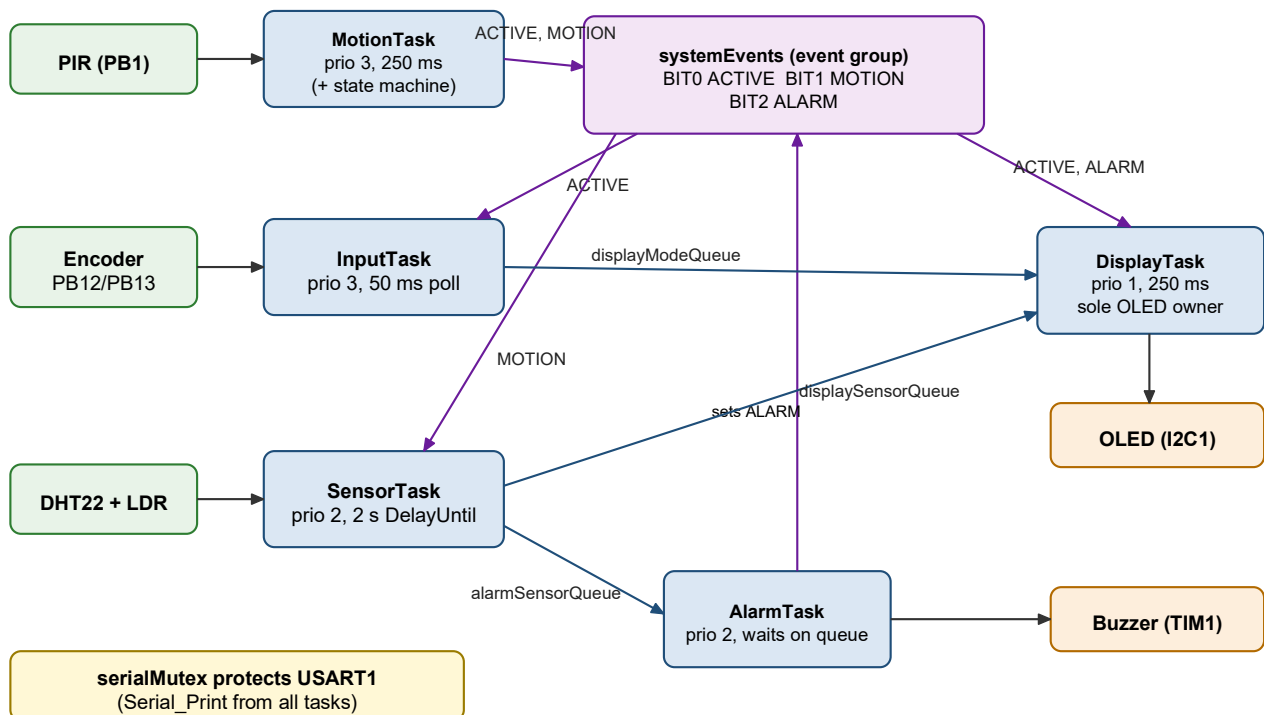


Figure 4. Task communication. Blue = queues, purple = event group bits.

Task	Job	Trigger / period	Priority	IPC	Usually blocked on
MotionTask	Read PIR, track timeout, set state	250 ms	3	Sets ACTIVE and MOTION bits	vTaskDelayUntil
InputTask	Encoder to page	50 ms	3	displayModeQueue; reads ACTIVE	vTaskDelay
SensorTask	DHT22 + light	2 s	2	Writes both sensor queues; reads MOTION	vTaskDelayUntil

AlarmTask	Temperature check, buzzer	New reading	2	alarmSensorQueue; sets ALARM	xQueueReceive
DisplayTask	Draw OLED	Every 250 ms	1	Two queues + event group	vTaskDelay

Priorities. MotionTask and InputTask are priority 3 because they react to things outside the system. If motion is handled late, the system stays asleep while someone is in the room, and a slow encoder feels unresponsive. Both do very little work and block often, so they don't starve the others. SensorTask and AlarmTask are priority 2 because they work on a 2 second cycle and a small delay is fine for room temperature. DisplayTask is priority 1 because OLED transfers over I2C are slow and it's better for the screen to update a bit late than for input or sensing to be delayed.

Task states. A task is Running while it does its work, Blocked while it waits in vTaskDelay, vTaskDelayUntil or xQueueReceive (and uses no CPU), and Ready when it could run but a higher priority task is running. For example, SensorTask is Running while it reads the DHT22, Blocked during its 2 second wait, and Ready if MotionTask is running when its wait ends. When all tasks are blocked the idle hook sleeps with __WFI(). I did not use Suspended or Deleted because all tasks run for the whole program.

vTaskDelayUntil vs vTaskDelay. vTaskDelay(2000) waits 2 seconds starting from when it is called, which is after the sensors were already read. So each cycle is 2 seconds plus the reading time, and that extra time adds up so the readings slowly drift. vTaskDelayUntil waits until a fixed time (last wake time + 2 s), so SensorTask always wakes at 0, 2, 4, 6 s. I also used it in MotionTask for its 250 ms period.

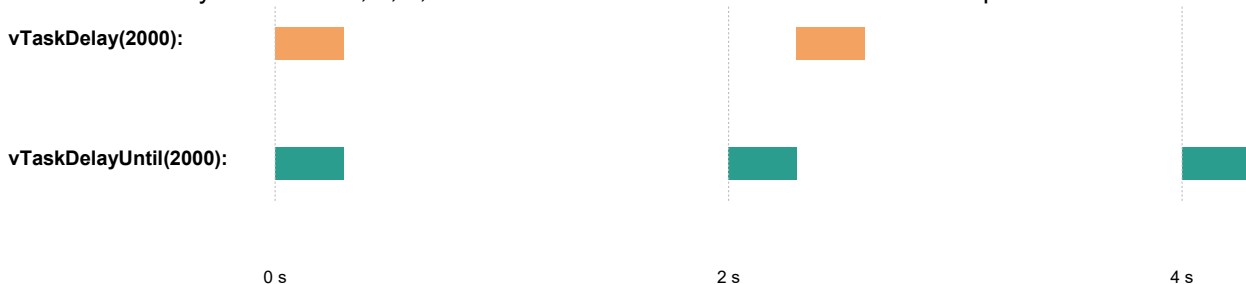


Figure 5. With a 300 ms read, vTaskDelay drifts but vTaskDelayUntil stays on a 2 s schedule.

Queues.

Queue	From / to	Data
displaySensorQueue	SensorTask to DisplayTask	SensorData
alarmSensorQueue	SensorTask to AlarmTask	SensorData
displayModeQueue	InputTask to DisplayTask	DisplayMode

SensorData holds temperature, humidity, light level, motion and a dhtValid flag. I used queues instead of global variables because the queue copies the whole struct at once, so a task can't read it halfway through an update. AlarmTask also just blocks on its queue until new data comes, so it doesn't poll. All queues have length 1 and use xQueueOverwrite so the reader always gets the newest value. I needed two sensor queues because a FreeRTOS queue is not a broadcast: if DisplayTask and AlarmTask shared one, whichever read first would take the item from the other.

Event group.

Bit	Name	Set / cleared by	Read by	Meaning
0	EVENT_ACTIVE	MotionTask (set at start and on motion, cleared on timeout)	DisplayTask, InputTask	System is ACTIVE
1	EVENT_MOTION	MotionTask (follows the PIR pin)	SensorTask	PIR is high
2	EVENT_ALARM	AlarmTask	DisplayTask	Temperature alarm is on

I used an event group because the state has to be seen by more than one task at the same time, which a queue can't do.

Mutex. The shared resource is USART1, the serial port. SensorTask, DisplayTask, InputTask, MotionTask and AlarmTask all print to it with Serial_Print(). A task can be switched out while the UART is still sending characters, so without protection two messages could mix into one unreadable line (for example a sensor reading cut in half by an encoder message). Serial_Print() takes serialMutex, writes the whole string, then gives it back, so each message comes out complete. The mutex protects one call at a time, so messages that must stay together are built into one string first. Before the scheduler starts and in fault hooks I use Serial_WriteRaw() instead, because the mutex can't be used there.

4. Implementation

Start-up. main() sets up the serial port early so boot messages show even if something fails, then runs Hardware_Init(), creates the queues, mutex and event group, creates the five tasks with their priorities and starts the scheduler.

DHT22. The DHT22 needs microsecond timing, so TIM4 runs as a 1 MHz counter. The start pulse is sent as an output, then the pin is switched to input with pull-up to read the 40 bits. Every wait has a timeout. The read is done inside a critical section so the tick can't break the timing. If the checksum fails, dhtValid is set to false and the alarm ignores that sample.

Light sensor. The ADC gives 0-4095, and the module's voltage goes up when it's darker, so LightPercentFromAdc() flips it: $(4095 - \text{raw}) \times 100 / 4095$. At first the direction was wrong, so I corrected the mapping and checked that brighter gives a higher number.

OLED. Only DisplayTask touches the OLED. Every 250 ms it checks the queues and event bits and redraws the page. When the system goes INACTIVE it clears the screen and turns the display off.

Encoder. InputTask reads CLK every 50 ms. On a falling edge of CLK it reads DT to decide the direction and calls ScrollNext() or ScrollPrev(). I kept 50 ms because faster polling made Wokwi slower.

Alarm. EvaluateTemperature() returns NORMAL, LOW_TEMPERATURE or HIGH_TEMPERATURE. Exactly 18 C and 30 C count as normal. AlarmTask only changes the buzzer when the state changes and sets or clears EVENT_ALARM.

Buzzer. The buzzer uses its own timer, TIM1 channel 1 on PA8, at 1 kHz. Buzzer_Set() changes the duty cycle between 0 and 50 %.

5. Verification and Testing

Unit tests

I wrote 15 unit tests with Unity and ran them on my PC with `pio test -e native`. All 15 passed.

Group	Test	Expected
Alarm	Below 18 C	LOW_TEMPERATURE
Alarm	Exactly 18 C	NORMAL
Alarm	Inside range	NORMAL
Alarm	Exactly 30 C	NORMAL
Alarm	Above 30 C	HIGH_TEMPERATURE
Light	Minimum ADC value	100 % (brightest)
Light	Maximum ADC value	0 % (darkest)

Navigation	Next from Temperature	Humidity
Navigation	Next from Motion	Temperature (wraparound)
Navigation	Previous from Humidity	Temperature
Navigation	Previous from Temperature	Motion (wraparound)
State	ACTIVE, no timeout	ACTIVE
State	ACTIVE, timeout	INACTIVE
State	INACTIVE, no motion	INACTIVE
State	INACTIVE, motion	ACTIVE

Wokwi functional tests

ID	What I did	Expected	What I saw	Result
FT-01	Set DHT22 to 28 C	Temperature updates	Log: [SensorTask] T=28.0 C H=61.2 % Light=75 % Motion=NO DHT=OK; OLED updated	PASS
FT-02	Changed DHT22 humidity	Humidity updates	Humidity page updated	PASS
FT-03	Changed LDR illumination	Light changes, brighter = higher	Light % followed the slider after I fixed the mapping	PASS
FT-04	Encoder clockwise	Next page	Next page selected, log '[Input] Dial: RIGHT'	PASS
FT-05	Encoder counterclockwise	Previous page	Previous page selected, log '[Input] Dial: LEFT'	PASS
FT-06	Temperature above 30 C	Alarm on	'[Alarm] HIGH TEMPERATURE', buzzer on, ALARM on OLED	PASS
FT-07	Temperature back to normal	Alarm off	'[Alarm] NORMAL', buzzer off	PASS
FT-08	PIR while ACTIVE	Stays ACTIVE	'[Motion] DETECTED', stayed ACTIVE	PASS
FT-09	No motion for 15 s	INACTIVE	'[Motion] INACTIVE: timeout', OLED off	PASS
FT-10	PIR while INACTIVE	ACTIVE again	'[Motion] ACTIVE: motion detected', OLED back on	PASS

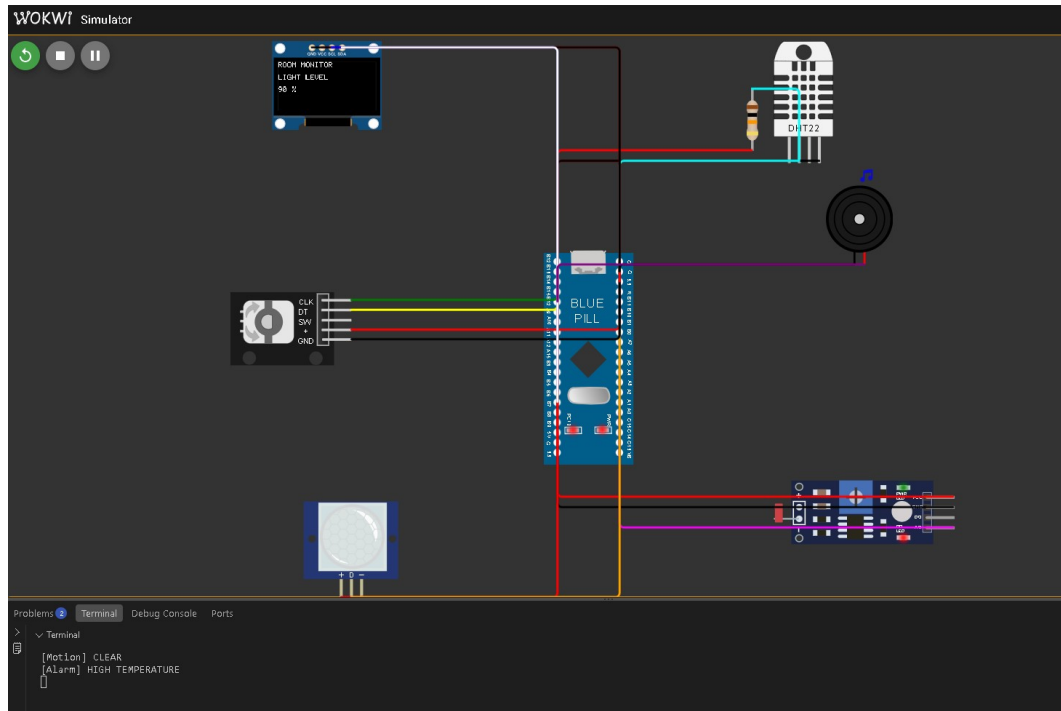


Figure 6. Finished system: temperature above 30 C, the buzzer is sounding and the log shows [Alarm] HIGH TEMPERATURE.

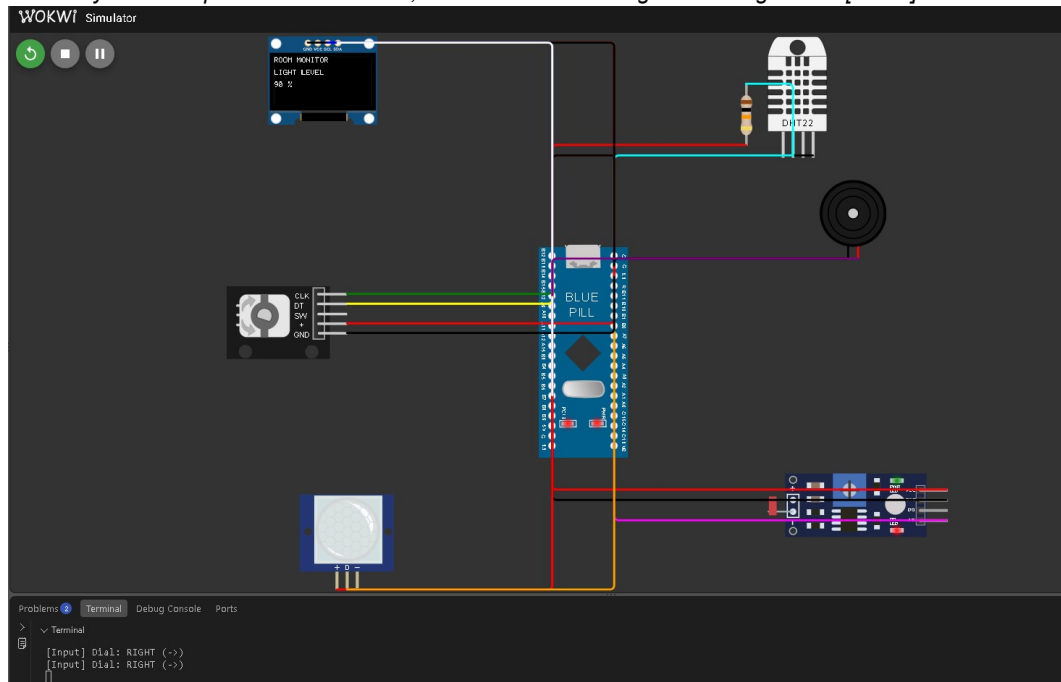


Figure 7. Turning the encoder: the OLED shows the Light Level page and the log shows [Input] Dial: RIGHT.

Fault experiments

Experiment	What I changed	What I saw	Why
1. No blocking	Removed the 50 ms vTaskDelay from InputTask	The OLED stopped responding, the log kept printing 'Dial: LEFT' without me turning it, and the buzzer seemed dead.	InputTask stayed Ready at priority 3 and kept the CPU busy. It also read the pins so fast that small changes looked like turns. The lower priority SensorTask, AlarmTask and DisplayTask were starved, so no new readings, no alarm updates and no screen refresh.
2. Higher priority	InputTask from 3 to 6	It stayed responsive while I turned the encoder.	InputTask still blocks for 50 ms and does very little work, so a higher priority didn't take CPU away. Priority only matters when a task is Ready. I put it back to 3.

3. No mutex	Bypassed serialMutex in Serial_Print()	Messages stayed readable and separate while I changed the LDR.	The messages are short and didn't happen to overlap. That doesn't mean the mutex is useless: if a task is switched out mid-message and another prints, the characters can mix. I put the mutex back.
-------------	--	--	--

6. Static Code Analysis

I ran `pio check` (cppcheck). It found no high or medium problems, only low style notes.

Finding	File	Cause	What I did
Duplicate condition	display.cpp	The same sign check was written twice when formatting the value	Fixed: calculate the value once and add the sign separately. Gone in the next run
Unused function	hardware.cpp: Hardware_VectorTableOk()	It really wasn't used	Removed it
Unused function	Tasks, EvaluateTemperature, ScrollNext/Prev, Hardware_Init	Called from other files or through xTaskCreate, which cppcheck doesn't follow	Kept, not a real problem
Unused function	stm32f1xx_it.c, hal_msp.c, hal_timebase_tim.c	Interrupt handlers and HAL callbacks are called by the vector table / HAL	Kept
Pointer could be const	stm32f1xx_hal_msp.c	HAL callback signatures are fixed	Kept so they still match HAL
C-style cast	hardware.cpp	Comes from STM32 register macros	Kept, vendor code

7. Engineering Discussion

Problems I ran into

Problem	What I did
FreeRTOS tasks would not run in Wokwi with the normal port	Used the Wokwi-compatible FreeRTOS V10.3.1 port from classmate Ni-ear. It uses TIM3 for the tick and switches tasks without SysTick/SVC/PendSV. I also used Ni-ear's linker script and startup approach in hardware.cpp.
Freezes and priority problems early on	Set the NVIC priority grouping and interrupt priorities in main() and fixed a UART overflow
Buzzer was silent on PB10 (TIM2 CH3, shared with the HAL time base)	Moved it to its own timer, TIM1 CH1 on PA8
Light % went the wrong way	Flipped the ADC mapping and added unit tests for both ends
Encoder lag in Wokwi	Kept 50 ms polling; faster polling made the simulation slower

Trade-offs

I polled the encoder and PIR instead of using interrupts. It is simpler and fast enough here, but on real hardware EXTI interrupts would respond faster and not miss quick turns. I let MotionTask handle the state machine instead of adding a StateTask, which means one less task but MotionTask does two jobs. DisplayTask checks every 250 ms instead of blocking on the queue, which is simpler but does a little extra work. The queues only keep the newest reading, which is fine because only the current room condition matters.

Limitations

The FreeRTOS port only works in Wokwi; on a real Blue Pill the normal ARM_CM3 port should be used. The 20 Hz tick gives 50 ms timing resolution. The light value isn't calibrated. The DHT22 read runs inside a critical section. The 15 s timeout is short on purpose. Everything was only tested in simulation.

Requirements traceability

Requirement	Implementation	Verification
FR-01 Temperature	SensorTask	FT-01
FR-02 Humidity	SensorTask	FT-02
FR-03 Light	SensorTask, LightPercentFromAdc()	Light unit tests, FT-03
FR-04 Motion	MotionTask	FT-08, FT-10
FR-05 OLED	DisplayTask	FT-01 to FT-05
FR-06 Encoder	InputTask, ScrollNext(), ScrollPrev()	Navigation unit tests, FT-04, FT-05
FR-07 Alarm	AlarmTask, EvaluateTemperature()	Alarm unit tests, FT-06, FT-07
FR-08 Activity state	MotionTask, EVENT_ACTIVE	State unit tests, FT-08 to FT-10
FR-09 Auto inactive	MotionTask, UpdateSystemState()	State unit test, FT-09
FR-10 Auto reactivate	MotionTask, UpdateSystemState()	State unit test, FT-10

8. Conclusion

The system meets all 10 requirements with five FreeRTOS tasks, three queues, an event group, a mutex for the serial port and vTaskDelayUntil for periodic sampling. All 15 unit tests and all 10 Wokwi tests passed, and static analysis found no serious problems.

The fault experiments taught me the most. A task that doesn't block can starve everything below it, a higher priority doesn't matter much if the task is usually blocked, and the mutex is needed even if collisions are rare. Next time I would try it on a real Blue Pill with the normal FreeRTOS port, use interrupts for the encoder and PIR, and record exact numbers during every Wokwi test.

References

BCA182 Laboratory Activity No. 1, Asst. Prof. Paul Rodolf P. Castor, MSU-IIT. FreeRTOS Kernel V10.3.1 and FreeRTOS documentation. Wokwi-compatible FreeRTOS port, linker script, and hardware start-up approach by classmate Shaqkobe Dos Tejada "Ni-ear" (github.com/Ni-ear/bca182-freertos-multisensor). Assistance in troubleshooting the issue with the buzzer not working by classmate Shanice Reih Tanque. STM32CubeF1 HAL and CMSIS. PlatformIO, Unity and cppcheck. Wokwi docs (docs.wokwi.com). AI tools were used as an assistant for debugging and writing; I built, ran and tested the project myself.